Fine-tuning BERT for emotion classification

Max Tulley and Bridget Valdez
COMP 331 - Natural Language Processing
Prof. Meiqing Zhang
12/10/2025

1 Introduction

Text classification is a highly studied and fundamental problem in natural language processing (NLP). In 2018, Google open-sourced BERT (bidirectional encoder representations from transformers) which offered a novel and highly performant solution to text classification. As the name suggests, BERT attends to context both before and after the current token, unlike early GPT models for example which only attend to context before the current token. To illustrate why this is important, the example of distinguishing between ‘money bank’ and ‘river bank’ is often used. Context both before and after the word ‘bank’ can tell BERT its intended meaning.

BERT models are pretrained to be generally capable of a wide range of NLP tasks. Fine-tuning can improve BERT’s performance on specific tasks, like classification. Fine-tuning is the process of training a pretrained model on a smaller task-specific dataset. This paper explores methods for fine-tuning BERT and proposes several specific practices which optimize BERT for multilabel emotion classification.

2 Dataset

Our dataset comes from a paper (Saravia et al., 2018) [2] and is published on HuggingFace (link to dataset). Conveniently, the dataset comes preprocessed and in a pandas dataframe. The dataset includes 16,000 training examples, 2,000 validation examples and 2,000 testing examples. Each labeled example consists of text and a class label which

corresponds to an emotion. The texts are all statements which begin with ‘I’ and describe an emotional state. For example, “I am feeling grouchy” is labeled with 3, corresponding to class 3: anger. The classes are: “anger”, “joy”, “sadness”, “surprise”, “love” and “fear”.

3 Methods

The methods discussed in this paper are: (1) defining head layers, (2) freezing pretrained layers, (3) implementing class weights, (4) padding to a proper length, (5) adjusting hyperparameters.

3.1 Head Layers

The method of fine-tuning explored in this paper is that of training newly defined head layers on a small task-specific dataset. The additional layers, in order, are as follows:

1. Linear
2. ReLu
3. Dropout
4. Linear

This is a common layer architecture for classification tasks. Head layer architecture is discussed by Houlsby et al [3].

Freezing layers refers to turning off the gradient calculations for layers of the model which prevents the model from changing the weights of those layers. We decided to freeze the pretrained layers in order to keep BERTs general capabilities that are trained into the model [4]. We chose to only train the newly defined head layers described above, which also reduces training time.

3.2 Class Weights

Data preprocessing for this dataset has already been done, however, care must still be taken in order to prevent the model from hyperoptimizing for only the training data. Each class in the training dataset contains a different number of labeled examples as shown in Figure 1.

Figure 1: Screenshot of Training Dataset Class Distribution

This can cause the model to develop higher weights towards classes with more examples. One strategy to prevent this problem is to pass class weights to the loss function which balances the learned weights for an uneven training dataset. Sometimes, however, the uneven distribution of classes in the training dataset might reflect a natural distribution encountered in the testing dataset and in real-world data. We tested the model's performance with and without class weighting (see results section).

3.3 Padding Length

It is important to pad each example text to the same total length so that weights can be computed efficiently through parallel computing. Choosing a good padding length is important since a padding length that is too small might cut out important information in your training texts, while one that is too large increases the computational cost of training. In order to find a good length we visually chose a padding length that would include all of the tokens for most of the training examples (see Figure 2). We chose a padding length of 50.

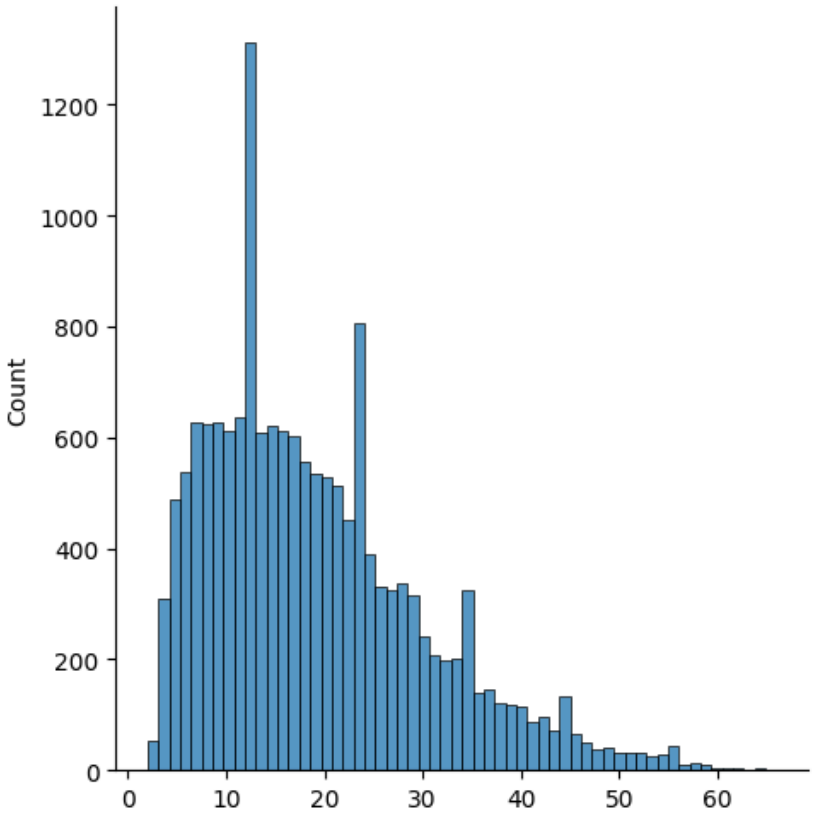

Figure 2: Screenshot of Training Dataset Token Lengths (horizontal-axis: number of tokens, vertical-axis: number of examples in training dataset with certain number of tokens)

3.4 Hyperparameters

The hyperparameters which we adjusted and tested during the training of our model are: (1) learning rate and (2) training epochs. We started with a learning rate of 2e-5, recommended

by Devlin et al [2], and manually tested similar rates. For training epochs, the standard is one to four epochs so we only needed to test four times, which we did manually.

4 Evaluation

To assess the performance of our fine-tuned model we utilized several different metrics. Precision is used to quantify the proportion of true positive predictions out of all positive predictions for a given class. Precision allows us to understand how reliable our models' positive predictions are. Precision is given by: $\frac{TP}{TP + FP}$ where TP = True Positives and FP = False Positives. High precision indicates that if a model predicts a label for given sentiment it is likely to be correct.

Recall is used to measure the proportion of actual positive instances that the model correctly classified. Recall is given by: $\frac{TP}{TP+FN}$ where TP = True Positives and FN = False Negatives. Recall gives us insight as to how many actual instances of a class were classified correctly. High recall indicates that the model successfully classifies most instances of a given class.

F1-score is the harmonic mean of precision and recall. F-1 allows us to understand how successful the model is, balancing both precision and recall. The harmonic mean ensures that both precision and recall need to be high for a high F-1 score. F1 is given by: $2 \times \frac{Precision \times Recall}{Precision + Recall}$. F-1 scores are particularly useful for our sentiment analysis because it allows us to see how our model is performing, taking into account a potential poor performance in a minority class.

Lastly, we looked at support, which is the number of actual correct occurrences in a given class. Support is useful because it allows us to understand if metrics like precision and recall are reliable for underrepresented classes. High scores on minority classes may be

misleading if there are few examples. Support helps identify if observed performance in a class is statistically meaningful or a product of sample size.

For visualization we generated confusion matrices, which provide the actual versus predicted classes. Confusion matrices provide insight into which incorrect classes the model often chooses for each actual class. This can provide valuable information into the model's understanding, since confusing two similar classes shows that the model holds a better understanding than confusing two distinct classes, or worst of all confusing all other classes equally and more.

5 Results

This section discusses our findings after testing with different hyperparameters and weighted classes.

5.1 Weighted Classes

Weighted classes resulted in a minor improvement to test accuracy as seen in Figure 3.

	Weighted Classes	Unweighted Classes
Test Accuracy	**0.9255**	0.9230

Figure 3: Table of Weighted Classes Versus Unweighted Classes Accuracies

This suggests that the distribution of classes in the training dataset closely resembles the distribution of the testing dataset. Assuming that the testing dataset accurately reflects the distribution of real-world data, then using weighted versus unweighted classes for our purpose does not make much of a difference. Despite this, for the small increase in accuracy, we used weighted classes for the rest of our testing.

5.2 Hyperparameters

Epochs were tested on a learning rate of 2e-5 with weighted classes.

Epochs	1	2	3	4
Test Accuracy	0.8800	0.9160	0.9160	**0.9255**

Figure 4: Table Showing Accuracy for Different Numbers of Epochs

Learning rates were tested with 4 epochs and weighted classes.

Learning Rate	2e-3	2e-4	2e-5	2e-6
Test Accuracy	0.3475	0.2905	**0.9235**	0.6505

Figure 5: Table Showing Accuracy for Different Learning Rates

The dramatic difference between learning rates suggests that we should have tested on a much smaller difference between the base learning rate of 2e-5. Clearly, 2e-5 is very near the optimal learning rate.

6 Conclusion

We found that 4 epochs and a learning rate of 2e-5 resulted in the best accuracy. We also found that weighted classes did not make a huge difference in accuracy.

In examining the confusion matrix generated from testing the model on the optimal hyperparameters, we found that the model confused similar emotions. For example, the model predicted ‘joy’ for actual ‘love’ examples and vice versa more often than it predicted other labels for those classes. Similarly the model commonly predicted ‘surprise’ for actual ‘fear’ examples and vice versa. This suggests that even when the model predicts the wrong label, it will often choose a similar emotion which highlights its deep understanding of the emotional classes.

Potential limitations in our fine-tuning may lie in our data. The majority of our data was labeled joy and sadness while surprise made up a small percent. This distribution is reflected in our per class performance. Joy and sadness consistently had the highest F-1 score while surprise was fairly low. While this could also be attributed to ambiguity associated with a sentiment like surprise, our data still could be responsible.

Therefore, to extend this project it could be interesting to explore methods to address minority classes. Some potential methods worth exploring are oversampling or even different methods of class weighting. Oversampling increases the number of samples by duplicating existing samples or generating new ones using data augmentation such as SMOTE. While we explored one method of class weighting it could be worthwhile to see the effects of other methods like weighted random samplers. These methods could potentially improve our models accuracy in minority classes. It could also be valuable to explore other hyperparameters like warm up, weight decay, or even different BERT models. We haven't explored these parameters much and it could be interesting to learn what ways they may influence the success of our model. Additionally, seeing how BERT compares to roBERTa or another more specific model in sentiment analysis could be interesting to observe.

Fine-tuning language models like BERT for sentiment classification has a wide range of practical applications. While it is useful to be able to distinguish a good review from a bad review, identifying a more specific emotion offers a deeper understanding of user sentiment. Classifying social media reactions to a movie, show, or video game to a specific emotion allows companies and creators to better understand user responses. There are a number of applications that could benefit from a sentiment analysis model. Our results of fine-tuning BERT for sentiment analysis shows our model is able to accurately classify emotions; with further improvements this offers a viable solution to emotional classification tasks.

7 Code Availability

Our code can be found on GitHub (https://github.com/mtulley/emotion-classifier). Refer to the Readme document for setup and documentation.

References

1. Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.
2. Elvis Saravia, Hsien-Chi Toby Liu, Yen-Hao Huang, Junlin Wu, and Yi-Shin Chen. 2018. CARER: Contextualized Affect Representations for Emotion Recognition. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3687–3697, Brussels, Belgium. Association for Computational Linguistics.
3. Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., de Laroussilhe, Q., Gesmundo, A., Attariyan, M., & Gelly, S. (2019). Parameter-Efficient Transfer Learning for NLP. arXiv:1902.00751. https://arxiv.org/abs/1902.00751
4. Jian Gu, Aldeida Aleti, Chunyang Chen, and Hongyu Zhang. 2025. A Semantic-Aware Layer-Freezing Approach to Computation-Efficient Fine-Tuning of Language Models. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 8019–8033, Vienna, Austria. Association for Computational Linguistics.